Pattern Database Heuristics

Comprehensive Guide to PDB Implementation

Understanding Projection-Based Abstraction Heuristics

November 11, 2025

Contents

1 Introduction and Motivation **3**
- 1.1 Why Pattern Database Heuristics? . . . 3
- 1.2 What Makes PDBs Special? . . . 3

2 Formal Definition: Projections and PDB Heuristics **4**
- 2.1 The Projection Mapping . . . 4
- 2.2 Pattern Database Terminology . . . 4

3 Example: Logistics Problem **5**
- 3.1 Problem Setup . . . 5
- 3.2 Projection 1: Package Only . . . 5
- 3.3 Projection 2: Package and Truck A . . . 6

4 Theoretical Properties **7**
- 4.1 Larger Patterns Yield Better Heuristics . . . 7
- 4.2 The Trade-off . . . 7

5 Computing Pattern Databases **8**
- 5.1 Two-Phase Process . . . 8

6 Phase I: Precomputation **9**
- 6.1 The Challenge . . . 9
- 6.2 The Solution: Syntactic Projection . . . 10
- 6.3 The Complete Algorithm . . . 11
- 6.4 Why This Works . . . 11

7 Detailed Example: Computing a PDB **12**
- 7.1 Problem Definition . . . 12
- 7.2 Step 1: Syntactic Projection to $P = \{p\}$ . . . 13
- 7.3 Step 2: Build Abstract State Space . . . 14
- 7.4 Step 3: Compute h^* Values Using Backward Dijkstra . . . 15

8 Phase II: Lookup During Search **16**
8.1 Basic Observations 16
8.2 The Lookup Method 16
8.3 Perfect Hash Function for Patterns 17
8.4 Example: Hash Function for Package and Truck A 18

9 Summary and Key Takeaways **19**
9.1 What We Learned About PDBs 19
9.2 The PDB Pipeline 19
9.3 Advantages of Pattern Databases 20
9.4 Limitations and Challenges 20
9.5 Research Directions 20

1 Introduction and Motivation

1.1 Why Pattern Database Heuristics?

Pattern databases (PDBs) represent a concrete and powerful method for designing abstraction functions α and computing their associated heuristic functions. They are one of the most successful classes of abstraction heuristics in both planning and game-playing domains.

Key Reasons to Study PDBs

Why Pattern Databases Matter:

- **Most commonly used** class of abstraction heuristics outside planning (especially in games)
- **One of the most successful** classes of abstraction heuristics in planning
- **Active research area** from inception to present day (theoretical properties, implementation, pattern selection)
- **Most effective admissible heuristics** currently known for many search problems

1.2 What Makes PDBs Special?

Unlike general abstraction heuristics, PDBs are based on a specific class of abstraction mappings called **projections**. This restriction to projections provides:

- A systematic way to construct abstractions
- Efficient computation methods
- Strong theoretical guarantees
- Practical scalability

2 Formal Definition: Projections and PDB Heuristics

2.1 The Projection Mapping

Projection and PDB Heuristic

Let $\Pi = (V, \mathcal{A}, \text{cost}, I, G)$ be an FDR planning task with state space $\Theta_\Pi = (S, \mathcal{A}, \text{cost}, T, I, S_G)$, and let $P \subseteq V$.

For a partial state φ to V, by $\varphi|_P$ we denote the **restriction** of φ to P.

Let S^P be the set of partial states over P. The **projection** $\pi_P : S \mapsto S^P$ is defined by:

$$\pi_P(s) := s|_P$$

Key Insight:

π_P maps two states s_1 and s_2 to the same abstract state if and only if they **agree on all variables in** P.

2.2 Pattern Database Terminology

- **Pattern:** We refer to P as the **pattern** of π_P
- **PDB Heuristic:** The abstraction heuristic induced by π_P on Θ_Π is called a pattern database heuristic
- **Notation:** We write h^P as shorthand for h^{π_P}, and Θ_Π^P or Θ^P as shorthand for $\Theta_\Pi^{\pi_P}$
- **Pattern Database:** h^P is usually stored in a lookup table called a **pattern database (PDB)**

3 Example: Logistics Problem

3.1 Problem Setup

Consider a simplified logistics task with:

- One package
- Two trucks (A and B)
- Two locations (L and R)

State Variables:

- **package**: $\{L, R, A, B\}$ (at location L, at location R, in truck A, in truck B)
- **truck A**: $\{L, R\}$ (at location L or R)
- **truck B**: $\{L, R\}$ (at location L or R)

This creates a state space with $4 \times 2 \times 2 = 16$ possible states.

3.2 Projection 1: Package Only

Projection to P

Pattern: $P = \{\text{package}\}$

Abstract States: Only 4 abstract states

- L: package at location L
- R: package at location R (GOAL)
- A: package in truck A
- B: package in truck B

Abstraction Effect:

- State (L, L, R) maps to abstract state L
- State (L, R, R) also maps to abstract state L
- The truck positions are **ignored** in this abstraction

For the initial state LRR (package at L, truck A at R, truck B at R), we get:

$$h^{\{\text{package}\}}(\text{LRR}) = 2$$

This tells us we need at least 2 actions to get the package to location R.

3.3 Projection 2: Package and Truck A

Projection to Pattern P

Pattern: $P = \{\text{package}, \text{truck A}\}$

Abstract States: $4 \times 2 = 8$ abstract states

- (L, L): package at L, truck A at L
- (L, R): package at L, truck A at R
- (A, L): package in truck A, truck A at L
- (R, R): package at R, truck A at R (GOAL)
- etc.

Abstraction Effect:

- State (L, R, L) maps to abstract state (L, R)
- State (L, R, R) also maps to abstract state (L, R)
- Only truck B's position is **ignored** in this abstraction

For the same initial state LRR:

$$h^{\{\text{package,truck A}\}}(\text{LRR}) = 2$$

Observation

In this particular example, the heuristic values happen to be the same, but the second projection captures more information about the state space structure (it tracks truck A's position).

4 Theoretical Properties

4.1 Larger Patterns Yield Better Heuristics

One of the most important theoretical results about pattern databases:

Theorem: Larger Patterns Induce More Informative Heuristics

Let Π be an FDR planning task with variables V and states S, and let $P' \subseteq P \subseteq V$. Then:

$$h^P(s) \geq h^{P'}(s) \quad \text{for every state } s \in S$$

Proof Idea:

- $\pi_{P'}$ can be viewed as an abstraction of Θ^P
- The rest follows from admissibility of abstraction heuristics
- That is, h^* in $\Theta^{P'}$ is a lower-bound on h^* in Θ^P

Special Case: When $P = V$ (all variables), we have:

$$h^V = h^*$$

That is, the pattern database over all variables gives the perfect heuristic!

4.2 The Trade-off

Pattern Size Controls the Trade-off:

- **Larger patterns** $\rightarrow$ More accurate heuristics, but exponentially larger abstract state spaces
- **Smaller patterns** $\rightarrow$ Less accurate heuristics, but faster to compute and store

The size of the abstract state space is:

$$|\Theta^P| = \prod_{v \in P} |D_v|$$

This grows exponentially with $|P|$, making pattern selection a critical challenge.

5 Computing Pattern Databases

5.1 Two-Phase Process

Computing and using a pattern database involves two distinct phases:

Overview

Phase I: Precomputation

- Compute explicit graph representation of abstract state space $\Theta_{\Pi}^{\pi_P}$
- Compute abstract remaining cost for every abstract state
- Store in a lookup table (the PDB)

Phase II: Lookup During Search

- During search, use lookup in the PDB table to implement h^P
- Project current state s to abstract state $s|_P$
- Return stored value from PDB

6 Phase I: Precomputation

6.1 The Challenge

Let Π be a planning task and P a pattern. Let $\Theta = \Theta_\Pi$ and $\Theta' = \Theta_\Pi^{\pi_P}$.

We want to compute a graph representation of Θ'.

Why This Is Not Trivial

The Issue:

- Θ' is defined through a function on Θ:
  - Each concrete transition induces an abstract transition
  - Each concrete goal state induces an abstract goal state
- In principle, we could compute Θ' by iterating over all transitions/goal states of Θ
- **BUT:** This would take time $\Omega(\|\Theta\|)$
- This is equivalent to solving the original planning task using blind search!

Requirement:

We need a way of computing Θ' in time polynomial in $\|\Pi\|$ and $\|\Theta'\|$.

6.2 The Solution: Syntactic Projection

Syntactic Projection

Let $\Pi = (V, \mathcal{A}, \text{cost}, I, G)$ be an FDR planning task, and let $P \subseteq V$.

The **syntactic projection** of Π to P is the FDR planning task:

$$\Pi|_P = (P, \mathcal{A}|_P, c, I|_P, G|_P)$$

where $\mathcal{A}|_P := \{a|_P \mid a \in \mathcal{A}\}$ with:

- $\text{pre}_{a|_P} = (\text{pre}_a)|_P$
- $\text{eff}_{a|_P} = (\text{eff}_a)|_P$

Key Idea:

$\Pi|_P$ removes the variables outside P from all constructs in the planning task description Π.

Theorem: Syntactic Projection is Equivalent to Projection

Let Π be an FDR planning task, and let $P \subseteq V$. Then $\Theta_{(\Pi|_P)}$ is identical to $\Theta_\Pi^{\pi_P}$ except that actions a in $\Theta_\Pi^{\pi_P}$ correspond to actions $a|_P$ in $\Theta_{(\Pi|_P)}$.

Proof: Easy from definition.

The Payoff:

The state space of the syntactic projection is (modulo action renaming) the same as the abstract state space of the projection.

6.3 The Complete Algorithm

Algorithm 1 Computing Pattern Databases for FDR Tasks

1: **function** COMPUTEPDB($\Pi = (V, \mathcal{A}, \text{cost}, I, G)$, pattern $P \subseteq V$)
2: $\Pi' \leftarrow \Pi|_P$ ▷ Compute syntactic projection
3: Compute the reachable part of $\Theta_{\Pi'}$ by a complete forward search
4: In the explicit graph $\Theta_{\Pi'}$, add a new node x with a 0-cost incoming edge from every goal node
5: Run Dijkstra starting from x traversing edges backwards
6: ▷ This results in $h^*_{\Theta_{\Pi'}}$ for all abstract states in $\Theta_{\Pi'}$
7: **return** A table mapping abstract states to their $h^*_{\Theta_{\Pi'}}$-value
8: **end function**

Complexity:

This algorithm runs in time and space polynomial in $\|\Pi\| + \|\Theta_{\Pi'}\|$.

6.4 Why This Works

Line 1: Create syntactic projection (fast, just manipulates task description)

Line 2: Build abstract state space by forward search (only explores reachable abstract states)

Line 3: Add artificial goal node (technical trick for backward Dijkstra)

Line 4: Run Dijkstra backwards from goals (computes h^* for all abstract states)

Line 5: Return lookup table

7 Detailed Example: Computing a PDB

7.1 Problem Definition

Variables: $V = \{p, t_A, t_B\}$ with:

- $D_p = \{L, R, A, B\}$ (package location/status)
- $D_{t_A} = D_{t_B} = \{L, R\}$ (truck locations)

Actions: $\mathcal{A} =$

- $\{\text{pickup}(x, y) \mid x \in \{A, B\}, y \in \{L, R\}\}$
- $\{\text{drop}(x, y) \mid x \in \{A, B\}, y \in \{L, R\}\}$
- $\{\text{move}(x, y, y') \mid x \in \{A, B\}, y, y' \in \{L, R\}, y \neq y'\}$

Action Schemas:

- $\text{pre}_{\text{pickup}(x,y)} : t_x = y, p = y; \quad \text{eff}_{\text{pickup}(x,y)} : p = x$
- $\text{pre}_{\text{drop}(x,y)} : t_x = y, p = x; \quad \text{eff}_{\text{drop}(x,y)} : p = y$
- $\text{pre}_{\text{move}(x,y,y')} : t_x = y; \quad \text{eff}_{\text{move}(x,y,y')} : t_x = y'$

Initial State: $I = \{p = L, t_A = R, t_B = R\}$
Goal: $G = \{p = R\}$

7.2 Step 1: Syntactic Projection to $P = \{p\}$

After projecting to $P = \{p\}$:

Variables: $V = \{p\}$ with $D_p = \{L, R, A, B\}$

Actions: Same action names, but modified preconditions and effects:

- $\text{pre}_{\text{pickup}(x,y)} : p = y; \quad \text{eff}_{\text{pickup}(x,y)} : p = x$
- $\text{pre}_{\text{drop}(x,y)} : p = x; \quad \text{eff}_{\text{drop}(x,y)} : p = y$
- $\text{pre}_{\text{move}(x,y,y')} : \emptyset; \quad \text{eff}_{\text{move}(x,y,y')} : \emptyset$

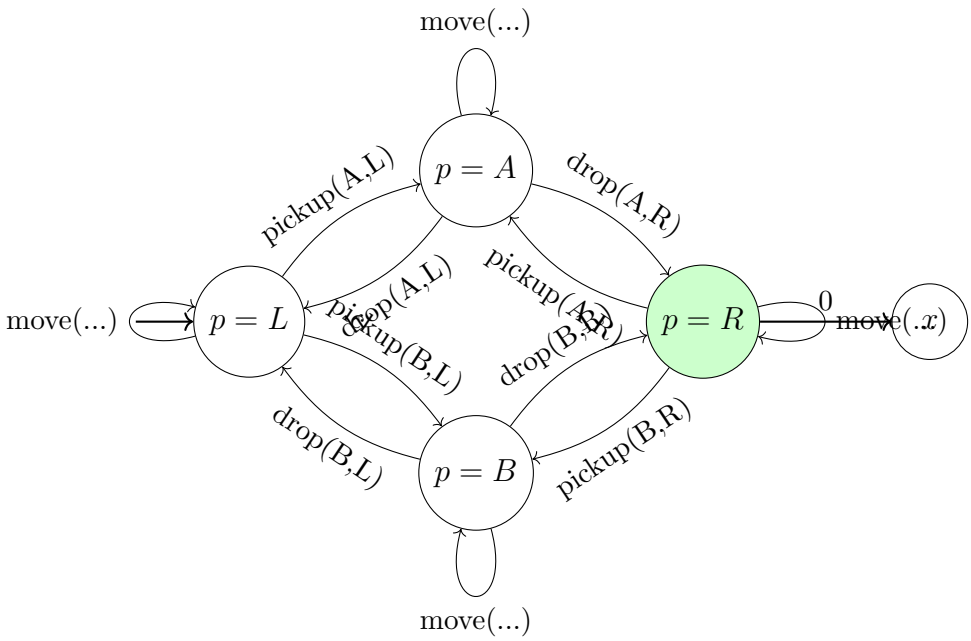
Initial State: $I = \{p = L\}$
Goal: $G = \{p = R\}$

Key Observation

The move actions now have empty preconditions and empty effects! They become no-ops in the abstract space. This is expected because moving trucks doesn't affect the package variable.

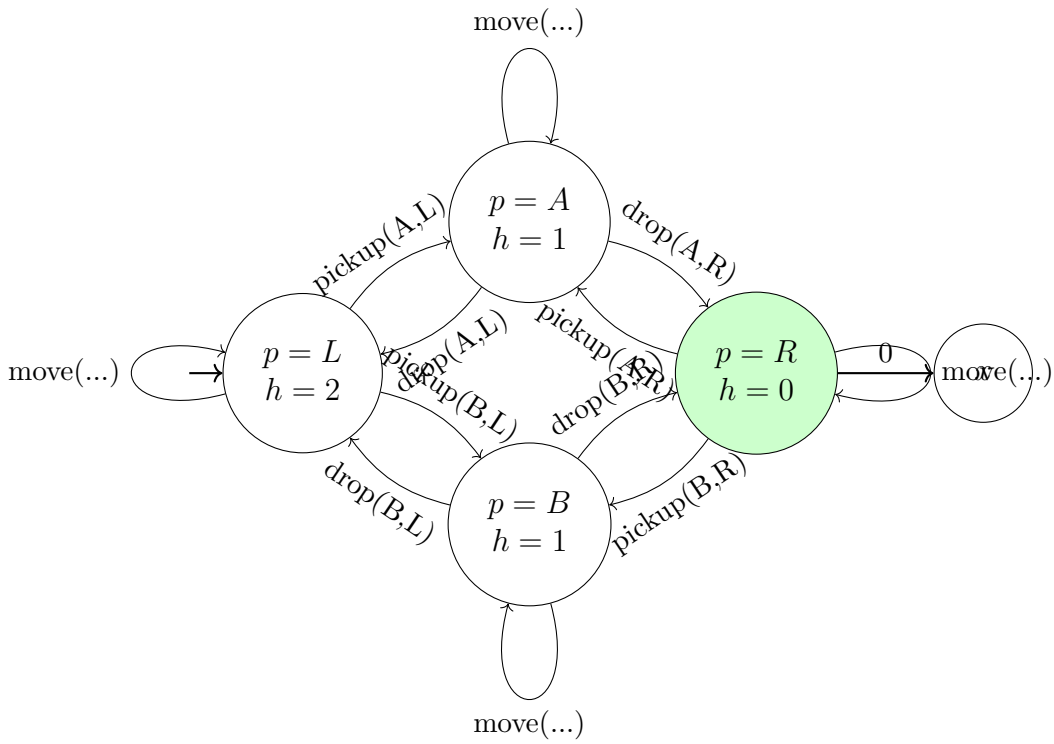
7.3 Step 2: Build Abstract State Space

The abstract state space $\Theta_{\Pi|_P}$ with added goal node x:



7.4 Step 3: Compute h^* Values Using Backward Dijkstra

After running Dijkstra backwards from node x:



Resulting Pattern Database

The pattern database for $P = \{p\}$:

Abstract State	L	A	B	R
h^P **value**	2	1	1	0

8 Phase II: Lookup During Search

8.1 Basic Observations

- During search, we **do not need** the actual abstract state space (transitions, etc.)
- The PDB is the **only piece of information** necessary to represent h^P
- We can **throw away** the abstract state space Θ' once the PDB is computed
- Space requirement during search is linear in $|S'|$: one table entry per abstract state

8.2 The Lookup Method

PDB Lookup Strategy

Design a perfect hash function:

- Map projected states $s|_P$ to numbers in range $\{0, \ldots, |S'| - 1\}$
- Index the PDB table by these hash values

During search:

1. Given a state s during search
2. Compute $h^P(s)$:
    - Map $\pi_P(s) = s|_P$ to its hash value
    - Lookup the table entry in PDB
    - Return the stored h^* value

8.3 Perfect Hash Function for Patterns

For a pattern $P = \{v_1, v_2, \ldots, v_k\}$ where variables are ordered, we use:

Pattern Hash Function

For each variable $v_i \in P$, define:

$$N_i = \prod_{j=1}^{i-1} |D_{v_j}|$$

Then the hash function is:

$$\text{index}(s) = \sum_{i=1}^{k} N_i \cdot s(v_i)$$

where $s(v_i)$ is encoded as an integer $0, 1, \ldots, |D_{v_i}| - 1$.

This is essentially treating the state as a number in a mixed-radix number system.

8.4 Example: Hash Function for Package and Truck A

Hash Function Example

Pattern variables and domains:

- $P = \{v_1, v_2\}$ with $v_1 =$ package, $v_2 =$ truck A
- $D_{v_1} = \{L, R, A, B\} \approx \{0, 1, 2, 3\}$
- $D_{v_2} = \{L, R\} \approx \{0, 1\}$

Compute multipliers:

- $N_1 = \prod_{j=1}^{0} |D_{v_j}| = 1$
- $N_2 = \prod_{j=1}^{1} |D_{v_j}| = |D_{v_1}| = 4$

Hash function:

$$\text{index}(s) = 1 \cdot s(\text{package}) + 4 \cdot s(\text{truck A})$$

Resulting Pattern Database:

abstract state	LL	RL	AL	BL	LR	RR	AR	BR
index	0	1	2	3	4	5	6	7
value	2	0	2	1	2	0	1	1

Usage example:

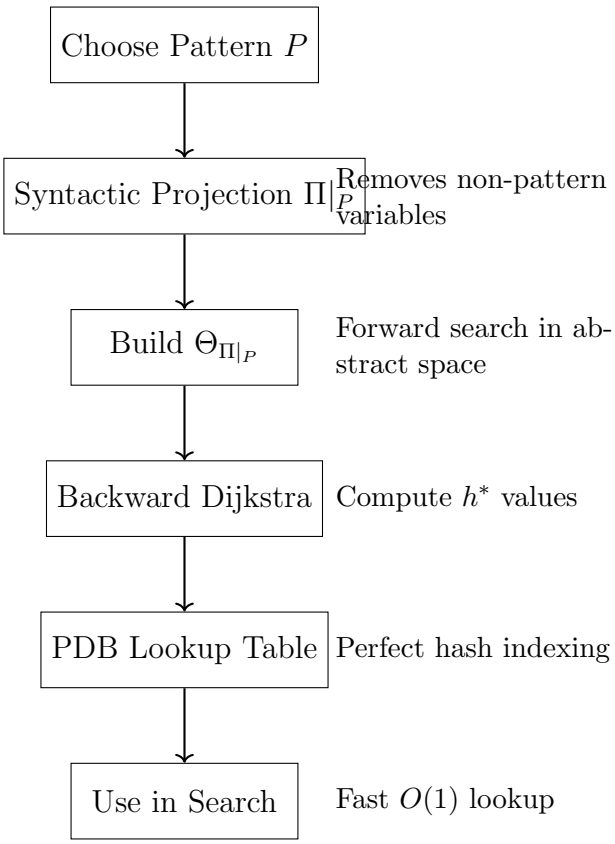
- State $s = (L, R, R)$ (package at L, truck A at R, truck B at R)
- Project to pattern: $s|_P = (L, R)$
- Encode: package=L=0, truck A=R=1
- Hash: index $= 1 \cdot 0 + 4 \cdot 1 = 4$
- Lookup: PDB[4] $= 2$
- Therefore: $h^P(s) = 2$

9 Summary and Key Takeaways

9.1 What We Learned About PDBs

1. **Definition:** PDBs are abstraction heuristics based on projection mappings $\pi_P : S \mapsto S^P$
2. **Pattern:** $P \subseteq V$ determines which variables are kept in the abstraction
3. **Larger patterns:** More informative heuristics but exponentially larger databases
4. **Computation:** Use syntactic projection to efficiently build abstract state space
5. **Storage:** Use perfect hash function to create compact lookup table
6. **Usage:** Fast constant-time lookup during search

9.2 The PDB Pipeline



9.3 Advantages of Pattern Databases

- **Admissible:** Always provide lower bounds on true cost
- **Efficient lookup:** Constant-time evaluation during search
- **Systematic:** Clear method for constructing abstractions
- **Flexible:** Can combine multiple PDBs (orthogonal patterns)
- **Proven success:** State-of-the-art for many domains

9.4 Limitations and Challenges

- **Exponential size:** Abstract state space grows as $\prod_{v \in P} |D_v|$
- **Memory intensive:** Must store entire lookup table
- **Pattern selection:** Choosing good patterns is non-trivial
- **Precomputation cost:** Building PDB can be expensive for large patterns

9.5 Research Directions

Active areas of PDB research include:

- Automatic pattern selection algorithms
- Combining multiple PDBs (additivity, orthogonal patterns)
- Compression techniques for large PDBs
- Symbolic PDB representations
- Online PDB construction during search

Next Steps

After mastering basic PDBs, the next topic is typically **Orthogonal Pattern Databases**, which allow us to combine multiple PDBs while preserving admissibility.